

HTTP in Detail

TryHackMe Room Report
Pre Security Learning Path

DAANIYA ABSAR

August 9, 2026

Abstract

This report documents my work through the **HTTP in Detail** room on TryHackMe, part of the *Pre Security* learning path. The room covers the fundamentals of the HyperText Transfer Protocol (HTTP), including the structure of requests and responses, common HTTP methods, status codes, headers, cookies, and a hands-on practical exercise interacting with a mock web application. The purpose of this report is to summarize the key concepts learned and demonstrate understanding of how clients and servers communicate over the web.

Contents

1	Introduction	2
2	Task 1: What is HTTP(S)?	2
2.1	Overview	2
2.2	Anatomy of a URL	2
3	Task 2: Requests and Responses	3
3.1	Example Request	3
3.2	Example Response	3
3.3	Breaking Down the Response	3
4	Task 3: HTTP Methods	4
4.1	Example: Creating a User	4
4.2	Example: Deleting a User	4
5	Task 4: HTTP Status Codes	4
6	Task 5: Headers	5
6.1	Common Request Headers	5
6.2	Common Response Headers	5
7	Task 6: Cookies	5
7.1	Example	5
8	Task 7: Making Requests (Practical)	6
8.1	Exercises Performed	6
9	Conclusion	6

1 Introduction

HTTP (HyperText Transfer Protocol) is the foundational protocol of the World Wide Web. It defines the rules by which web browsers (clients) and web servers communicate, allowing resources such as HTML pages, images, and data to be requested and delivered. Understanding HTTP is essential for anyone working in cybersecurity, as the vast majority of web-based attacks (e.g. SQL injection, XSS, IDOR, session hijacking) rely on manipulating HTTP requests and responses.

This room is broken down into the following tasks:

1. What is HTTP(S)?
2. Requests and Responses
3. HTTP Methods
4. HTTP Status Codes
5. Headers
6. Cookies
7. Making Requests (Practical)

2 Task 1: What is HTTP(S)?

2.1 Overview

HTTP stands for **HyperText Transfer Protocol**. It was developed by Tim Berners-Lee and his team between 1989 and 1991 and is the protocol that specifies how a web browser and a web server communicate.

HTTPS (HTTP **Secure**) is the encrypted version of HTTP. It uses TLS/SSL to encrypt data in transit, which:

- Prevents third parties from reading transmitted data (confidentiality).
- Provides assurance that the client is communicating with the legitimate server and not an impersonator (authenticity).

2.2 Anatomy of a URL

A URL (Uniform Resource Locator) is an instruction on how to access a resource on the internet. Example:

```
http://user:password@tryhackme.com:80/view-room?id=1#task3
```

Component	Description
http	The scheme – instructs the browser which protocol to use.
user:password	Optional credentials for basic authentication.
tryhackme.com	The host – domain name or IP address of the target server.
80	The port – typically 80 for HTTP and 443 for HTTPS.
/view-room	The path – the file or resource location being requested.
?id=1	The query string – extra parameters sent with the request.
#task3	The fragment – a reference to a location within the page.

Answer

What does HTTP stand for?

HyperText Transfer Protocol

What does the S in HTTPS stand for?

Secure

3 Task 2: Requests and Responses

Communication over HTTP follows a simple client-server model: the client sends a **request**, and the server replies with a **response**.

3.1 Example Request

```
GET /room HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
```

3.2 Example Response

```
HTTP/1.1 200 OK
Server: nginx/1.15.8
Date: Fri, 14 May 2021 18:35:09 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 233
Last-Modified: Fri, 14 May 2021 18:35:09 GMT

<html>
<head><title>TryHackMe</title></head>
<body>
Welcome to the Room page
</body>
</html>
```

3.3 Breaking Down the Response

- **Line 1:** The HTTP version (HTTP/1.1) followed by the status code (200 OK), indicating the request completed successfully.
- **Server:** Identifies the web server software handling the request.
- **Content-Type:** Tells the client what type of data is being returned (e.g. HTML, JSON).
- **Content-Length:** Tells the client how much data (in bytes) to expect, ensuring no data is missing.
- **Last-Modified:** When the resource was last changed.

Answer

What HTTP protocol version is being used in the example?

HTTP/1.1

What response header tells the browser how much data to expect?

Content-length

4 Task 3: HTTP Methods

HTTP methods (also called verbs) indicate the action the client wants to perform on a given resource.

Method	Purpose
GET	Retrieve information/data from the web server. Does not modify server state.
POST	Submit data to the server, often creating a new record (e.g. registering a user).
PUT	Submit data to update an existing resource.
DELETE	Remove a resource or record from the server.

4.1 Example: Creating a User

```
POST /user HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 14

username=admin
```

4.2 Example: Deleting a User

```
DELETE /user/1 HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
Content-Length: 4

id=1
```

Answer

What method would be used to create a new user account?
POST

5 Task 4: HTTP Status Codes

Status codes are three-digit numbers returned by the server that indicate the outcome of a request. They are grouped into five classes:

Range	Category
1xx	Informational – request received, continuing process.
2xx	Success – the request was successfully received, understood, and accepted.
3xx	Redirection – further action needs to be taken to complete the request.
4xx	Client Error – the request contains bad syntax or cannot be fulfilled (e.g. 404 Not Found).
5xx	Server Error – the server failed to fulfil a valid request (e.g. 500 Internal Server Error).

Note

A 5xx error typically indicates something has gone wrong on the server side – for example, if the application cannot reach its database and crashes.

Answer

What response code might you receive if the web server cannot access its database and the application crashes?

503

6 Task 5: Headers

Headers provide additional metadata about a request or response, allowing the client and server to exchange information beyond the basic message body.

6.1 Common Request Headers

- **Host:** Specifies the domain name of the server being requested. Since a single server can host multiple websites, this tells it which site is being requested (Virtual Hosting).
- **User-Agent:** Identifies the client software (browser, OS, version) making the request.
- **Cookie:** Sends previously stored cookie data back to the server.

6.2 Common Response Headers

- **Content-Type:** Describes the media type of the returned content (e.g. `text/html`, `application/json`).
- **Content-Length:** Size of the response body in bytes.
- **Set-Cookie:** Instructs the client to store a cookie for future requests.

Answer

Which header specifies the domain name being requested, allowing a server to host multiple sites?

HOST

Which header is used by a server to instruct the client to save a cookie?

SET COOKIE

7 Task 6: Cookies

Cookies are small pieces of data stored by the browser, sent by the server via the **Set-Cookie** response header. Once stored, the browser automatically includes them in the **Cookie** header on subsequent requests to the same domain.

7.1 Example

```
HTTP/1.1 200 OK
Set-Cookie: session_id=abc123; HttpOnly; Secure
```

Cookies are commonly used for:

- Session management (keeping a user logged in).
- Personalization (remembering preferences).
- Tracking (analytics, advertising).

Note

From a security perspective, cookies are a common attack target. Missing flags such as `HttpOnly` or `Secure` can expose sessions to theft via Cross-Site Scripting (XSS) or interception over unencrypted connections.

8 Task 7: Making Requests (Practical)

The final task involved interacting with a mock web application to practice sending real HTTP requests using different methods, and observing how the server responded.

8.1 Exercises Performed

1. Sent a `GET` request to `/room` and observed the response containing a success message.
2. Sent a `GET` request to `/blog?id=1` using a URI parameter to retrieve a specific blog post.
3. Practiced using `POST`, `PUT`, and `DELETE` requests against a mock user-management endpoint.
4. Observed how a web application proxy (e.g. Burp Suite) can be used to intercept and modify these requests – for example, changing cookie values or parameters before the request reaches the server.

Note

This practical demonstrates a foundational offensive security concept: if a server does not properly validate or authorize incoming requests, an attacker who intercepts and modifies a request (e.g. changing a user ID or role parameter) could perform unauthorized actions – this is the basis of vulnerabilities like **IDOR** (Insecure Direct Object Reference) and **Broken Access Control**.

9 Conclusion

Completing the **HTTP in Detail** room provided a solid foundation in how the HTTP protocol works, including:

- The difference between HTTP and HTTPS and why encryption matters.
- The structure of URLs and their individual components.
- How requests and responses are formatted and exchanged between client and server.
- The purpose of the four core HTTP methods: `GET`, `POST`, `PUT`, `DELETE`.
- How status codes communicate the outcome of a request.
- The role of headers in transmitting metadata (e.g. `Host`, `Content-Type`, `Set-Cookie`).
- How cookies enable stateful sessions on an inherently stateless protocol.
- Hands-on practice crafting and sending HTTP requests, and an introduction to how insecure request handling can lead to real vulnerabilities.

This knowledge forms a critical building block for further study in web application security, including topics such as authentication mechanisms, session management, and common web vulnerabilities (OWASP Top 10).

References

- TryHackMe – *HTTP in Detail* room: <https://tryhackme.com/room/httpindetail>
- TryHackMe – *Pre Security* Learning Path